26-1 프로젝트 수강신청서 및 계획서 내용

> 이 문서는 `25-1 프로젝트 수강신청서 및 계획서 - 양식.docx` 에 작성할 내용입니다.

1. 기본 정보

항목	내용
학번	C235180
성명	서민지
학과	컴퓨터공학과
지도교수	⚠️ **(교수님 성함 기재 필요)**
연락처	⚠️ **(이메일/전화번호 기재 필요)**

2. 프로젝트 제목

OS 스케줄링 알고리즘을 활용한 멀티테넌트 LLM API 요청 관리 시스템

영문: Multi-tenant LLM API Request Management System using OS Scheduling Algorithms

3. 프로젝트 개요

3.1 연구 배경

ChatGPT, Claude 등 대규모 언어 모델(LLM) API의 사용이 급증하면서, 다중 사용자 환경에서 API 요청을 효율적으로 관리해야 하는 필요성이 대두되고 있습니다. 현재 많은 LLM 서비스는 기본적인 선착순(FCFS) 처리나 단순 Rate Limiting에 의존하고 있어, 멀티테넌트 환경에서 다음과 같은 문제가 발생할 수 있습니다:

- **응답 지연**: 긴 요청이 짧은 요청을 지연시키는 Convoy Effect(호위 효과)
- **공정성 부재**: 모든 사용자가 동일하게 취급되어 VIP 사용자 차별화 불가
- **비용 비효율**: 우선순위 없이 처리하여 자원 낭비

운영체제 수업에서 프로세스 스케줄링을 학습하며 "실제 시스템에서도 이러한 알고리즘이 효과적일까?"라는 연구 동기를 갖게 되었으며, 최근 주목받는 LLM API 관리 분야에 적용하고자 본 주제를 선정하였습니다.

3.2 해결 방안

운영체제 수업에서 학습한 CPU 스케줄링 알고리즘을 LLM API 요청 관리에 적용합니다:

1. **FCFS (First-Come, First-Served)**: 기본 선착순 처리 (베이스라인)
2. **Priority Scheduling**: 긴급 요청 우선 처리 + Aging(노화)으로 기아(Starvation) 방지
3. **MLFQ (Multi-Level Feedback Queue)**: 짧은 요청 최적화, 동적 우선순위
4. **WFQ (Weighted Fair Queuing)**: 사용자별 가중치 기반 공정한 자원 배분

3.3 OS 수업 연관성

OS 개념	본 프로젝트 적용
프로세스	LLM API 요청
CPU 시간	API 응답 시간
Ready Queue	요청 대기열
Context Switch	요청 전환
Starvation(기아 현상)	장시간 대기 요청
Aging(노화 기법)	대기 시간에 따른 우선순위 상승

4. 목표 및 기대효과

4.1 개발 목표

목표	상세 내용	측정 방법
알고리즘 구현	4가지 스케줄링 알고리즘 구현	단위 테스트 통과

목표	상세 내용	측정 방법
REST API	요청 등록/조회/통계 API	curl 테스트
LLM 연동	Ollama 로컬 LLM 연동	실제 응답 확인
성능 비교	알고리즘별 성능 측정	실험 데이터 분석

4.2 기대효과

1. **학습 성과**
   - OS 스케줄링 이론을 실제 시스템에 적용하며 이론과 실무의 연결 경험
   - Node.js/Express 기반 백엔드 설계 및 개발 역량 향상
   - 실험 설계, 데이터 수집, 정량적 분석 등 연구 수행 역량 습득
2. **학술적 기여**
   - OS 스케줄링 이론의 실제 AI 시스템 적용 사례 제시
   - 4가지 알고리즘의 정량적 성능 비교 데이터 제공
   - Jain's Fairness Index를 활용한 공정성 정량 분석
3. **실용적 가치**
   - 긴급 요청의 우선 처리로 LLM API 서비스 응답성 향상
   - 멀티테넌트 환경에서 가중치 기반 공정한 자원 배분 (WFQ)
   - 소스코드 공개를 통해 LLM 게이트웨이 구축 시 참고 자료로 활용 가능

5. 기술 스택

5.1 개발 환경

분류	기술	버전	선정 이유
언어	JavaScript	ES2024	학습 용이성
런타임	Node.js	20 LTS	안정성
웹 프레임워크	Express.js	4.18	간결한 API
데이터 저장	JSON 파일	-	별도 패키지 불필요, 단순 구조
테스트	Jest	29.x	표준 프레임워크

분류	기술	버전	선정 이유
LLM	Ollama	최신	로컬 무료 실행

5.2 기술 선정 기준

- **단순성**: 학부생 1명이 한 학기 내 구현 가능
- **학습 가치**: OS 알고리즘 이해에 집중
- **실용성**: 실제 동작하는 시스템 구축

6. 개발 일정

주차	기간	주요 활동	산출물
1-2주	3/1 - 3/14	요구사항 분석, 설계	계획서, 설계 문서
3-4주	3/15 - 3/28	FCFS, Priority 구현	기본 스케줄러
5-6주	3/29 - 4/11	MLFQ, WFQ 구현	전체 스케줄러
7-8주	4/12 - 4/25	API 개발, 테스트	REST API, 테스트
9-10주	4/26 - 5/9	실험, 데이터 수집	성능 비교 데이터
11-12주	5/10 - 5/23	보고서 작성	최종 보고서
13주	5/24 - 5/31	발표 준비	PPT, 데모

7. 시스템 구조

7.1 전체 아키텍처

```
클라이언트 (curl/Postman)
        |
        v
Express.js REST API
  POST /api/requests       (요청 제출)
  GET  /api/requests/:id   (상태 조회)
  GET  /api/scheduler/stats(통계)
  POST /api/scheduler/switch(알고리즘 변경)
  GET  /api/health         (헬스체크)
  ... 외 4개 엔드포인트
        |
        v
Scheduler Manager
  +------+------+------+------+
  | FCFS |Priority|MLFQ | WFQ |
  +------+------+------+------+
        |
        v
메모리 큐 + JSON 파일 저장소
        |
        v
Ollama (로컬 LLM)
```

7.2 핵심 모듈

모듈	설명	예상 코드량
BaseScheduler	스케줄러 공통 인터페이스	~30줄
FCFSScheduler	선착순 스케줄러	~60줄
PriorityScheduler	우선순위 스케줄러 + Aging	~100줄
MLFQScheduler	다단계 피드백 큐	~150줄
WFQScheduler	가중치 기반 공정 큐	~120줄
스케줄러 합계		**~460줄**

모듈	설명	예상 코드량
기타 모듈	큐, 저장소, API, LLM 클라이언트	~540줄
전체 합계		**~1,000줄**

8. 알고리즘 설계

8.1 FCFS (First-Come, First-Served)

- **원리**: 먼저 도착한 요청을 먼저 처리합니다.
- **LLM 적용**: 모든 LLM 요청을 도착 순서대로 처리하는 베이스라인 알고리즘입니다.
- **장점**: 구현이 간단하고, 모든 요청을 동등하게 처리합니다.
- **단점**: Convoy Effect(호위 효과) — 긴 요청이 짧은 요청을 지연시킵니다.

8.2 Priority Scheduling

- **원리**: 우선순위가 높은 요청을 먼저 처리합니다.
- **LLM 적용**: 긴급한 LLM 요청(예: 실시간 고객 응대)을 일반 요청보다 먼저 처리합니다.
- **우선순위**: URGENT(3) > HIGH(2) > NORMAL(1) > LOW(0)
- **Aging(노화 기법)**: 5초 주기로 대기 중인 요청의 우선순위를 1단계 상승시켜 기아 현상을 방지합니다.

8.3 MLFQ (Multi-Level Feedback Queue)

- **원리**: 4개 큐(Q0-Q3)를 두고, 요청의 실행 시간에 따라 큐를 이동시킵니다.
- **LLM 적용**: 짧은 LLM 요청(간단한 질문)은 높은 큐에서 빠르게 처리하고, 긴 요청(문서 요약 등)은 낮은 큐로 이동시켜 짧은 요청의 응답성을 보장합니다.
- **Time Quantum**: Q0(1초) < Q1(3초) < Q2(8초) < Q3(무제한)
- **Boost**: 30초 주기로 모든 요청을 Q0으로 재배치하여 장기 대기를 방지합니다.

8.4 WFQ (Weighted Fair Queuing)

- **원리**: 테넌트별 가중치에 비례하여 자원을 배분합니다.
- **LLM 적용**: 유료 테넌트(Enterprise)의 LLM 요청이 무료 테넌트(Free)보다 가중치에 비례하여 빠르게 처리되도록 합니다.
- **가중치**: Enterprise(100) > Premium(50) > Standard(10) > Free(1)
- **공정성 측정**: Jain's Fairness Index를 활용하여 가중치 대비 서비스 차등화를 정량적으로 평가합니다.

9. 예상 성과

9.1 연구 질문 (Research Questions)

본 프로젝트는 사전에 목표 수치를 정하지 않고, 실험 결과를 관찰하여 분석하는 **탐구적 연구(exploratory research)** 방식으로 진행하며, 다음 연구 질문에 답하고자 합니다:

연구 질문	알고리즘	핵심 탐구 내용
RQ1	Priority	긴급(URGENT) 요청은 낮은 우선순위 요청보다 얼마나 빠르게 처리되는가?
RQ2	MLFQ	다양한 길이의 작업이 혼재된 환경에서 MLFQ는 어떤 적응성을 보이는가?
RQ3	WFQ	가중치에 비례하는 서비스 차등화를 달성하는가?

9.2 예상되는 결과

며칠간 간단한 프로토타입을 만들어 사전 테스트해 본 결과, FCFS 대비 Priority 스케줄링에서 URGENT 요청의 평균 대기시간이 약 81% 감소하는 양상을 확인하였습니다 (간이 프로토타입 결과이며, 본격적인 실험에서는 조건에 따라 달라질 수 있습니다). 이를 바탕으로 다음과 같은 결과를 기대합니다:

- **Priority Scheduling**: URGENT 요청이 FCFS 대비 의미 있게 빠르게 처리될 것으로 예상 (프로토타입에서 확인 완료)
- **MLFQ**: 짧은 요청과 긴 요청이 혼재된 환경에서 적응적 성능을 보일 것으로 예상
- **WFQ**: 테넌트 가중치에 비례하는 서비스 차등화가 관찰될 것으로 예상

9.3 품질 목표

항목	목표
테스트 커버리지	85%+
테스트 통과율	100%
코드 라인	스케줄러 ~460줄, 전체 ~1,000줄

9.4 프로젝트 한계점

- 본 프로젝트는 **로컬 환경에서의 개념 증명(PoC)** 수준으로, 실제 대규모 트래픽 환경에서의 검증은 포함하지 않습니다.
- Ollama 로컬 LLM 사용으로, 실제 클라우드 LLM API(OpenAI 등)와 응답 특성이 다를 수 있습니다.
- 단일 서버 환경에서의 실험이므로, 분산 환경에서의 확장성은 향후 과제로 남깁니다.

10. 위험 요소 및 대응 방안

위험	확률	영향	대응 방안
Ollama 설치/연동 실패	낮음	높음	공식 문서 참조, Mock 객체로 테스트 대체
알고리즘 구현 어려움	중간	중간	OS 수업 자료 참조, 단계별 점진적 구현
일정 지연	중간	중간	FCFS+Priority를 우선 구현하여 최소 결과물 확보

11. 참고 자료

1. Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating System Concepts* (10th ed.). Wiley.
2. Arpaci-Dusseau, R. H., & Arpaci-Dusseau, A. C. (2018). *Operating Systems: Three Easy Pieces*. Arpaci-Dusseau Books.
3. Demers, A., Keshav, S., & Shenker, S. (1989). "Analysis and simulation of a fair queueing algorithm". *ACM SIGCOMM Computer Communication Review*, 19(4), 1-12.
4. Jain, R., Chiu, D. M., & Hawe, W. R. (1984). "A quantitative measure of fairness and discrimination for resource allocation in shared computer systems". *DEC Research Report TR-301*.
5. Node.js Foundation. (n.d.). *Node.js Documentation*. https://nodejs.org/docs
6. StrongLoop/Express. (n.d.). *Express.js Documentation*. https://expressjs.com
7. Ollama. (n.d.). *Ollama Documentation*. https://ollama.ai/docs

12. 서약

본인은 위 계획서의 내용대로 졸업프로젝트를 성실히 수행할 것을 서약합니다.

작성일: 2026년 2월 8일

작성자: 서민지 (서명)

이 문서의 내용을 docx 양식에 복사하여 사용하세요.